Total no. of Pages:04 *Roll no……..*

3rd SEMESTER **B.Tech.**

END TERM EXAMINATION ***Nov-2024***

MC209: DATABASE MANAGEMENT SYSTEMS

Time: 03:00 Hours ***Max. Marks: 40***

Note : All questions carry equal marks. DO ANY 5
Assume suitable missing data, if any.

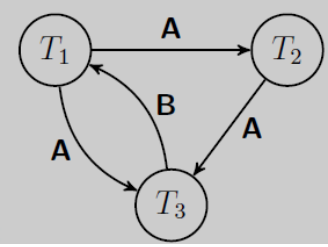
Q1. **[CO1], [BTL 5], [5+3]**

a. Consider the schedule given below. R(·) and W(·) stand for 'Read' and 'Write', respectively

time	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}	t_{11}
T_1	R(A)		W(A)						R(B)		W(B)
T_2				R(C)	R(A)		W(A)			W(C)	
T_3		R(B)				W(B)		R(A)			

i. Is this schedule serial? NO

ii. Give the dependency graph of this schedule.



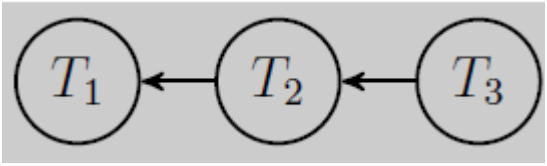
iii. Is this schedule conflict serializable?
NO
If "yes", provide the equivalent serial schedule. If "no", briefly explain why.

Solution: The schedule is not serializable because there are two cycles in the dependency graph: $T_1 \to T_2 \to T_3 \to T_1$ and $T_1 \to T_3 \to T_1$.

iv. Is this schedule view serializable? NO

b. Consider the following lock requests. Note that: S(·) and X(·) stand for 'shared lock' and 'exclusive lock', respectively. T1, T2, and T3 represent three transactions.

i. Give the wait-for graph for the lock requests at time-tick t7.



ii. Determine whether there exists a deadlock in the lock requests, and explain why.

A dead lock does not exists because there is no cycle in the dependency graph.

time	t_1	t_2	t_3	t_4	t_5	t_6	t_7
T_1	X(A)						S(C)
T_2			S(B)	S(C)		S(A)	
T_3		S(C)			X(B)		

Q2. **[CO4], [BTL 3], [5+3]**

a. **Consider the relation schema with attributes, E = {W, X, Y, Z} and the functional dependencies: XY → W, X → Z, W → X.**
   i. **List all the candidate key(s) for E. -** {W Y}, {XY}
   ii. **Is the relation E in 3NF? If yes, explain. If not, give a 3NF decomposition of E that is lossless, dependency preserving, and has as few tables as possible.**
   NO, X → Z violates and for every FD X → A, A is part of a candidate key. E1,1=(X, Z), E1,2=(W, X, Y)
   iii. **Is the relation E in BCNF? If yes, explain. If not, give a BCNF decomposition of E that is lossless, dependency preserving, and has as few tables as possible.**

NO, X → Z and W → X Violates

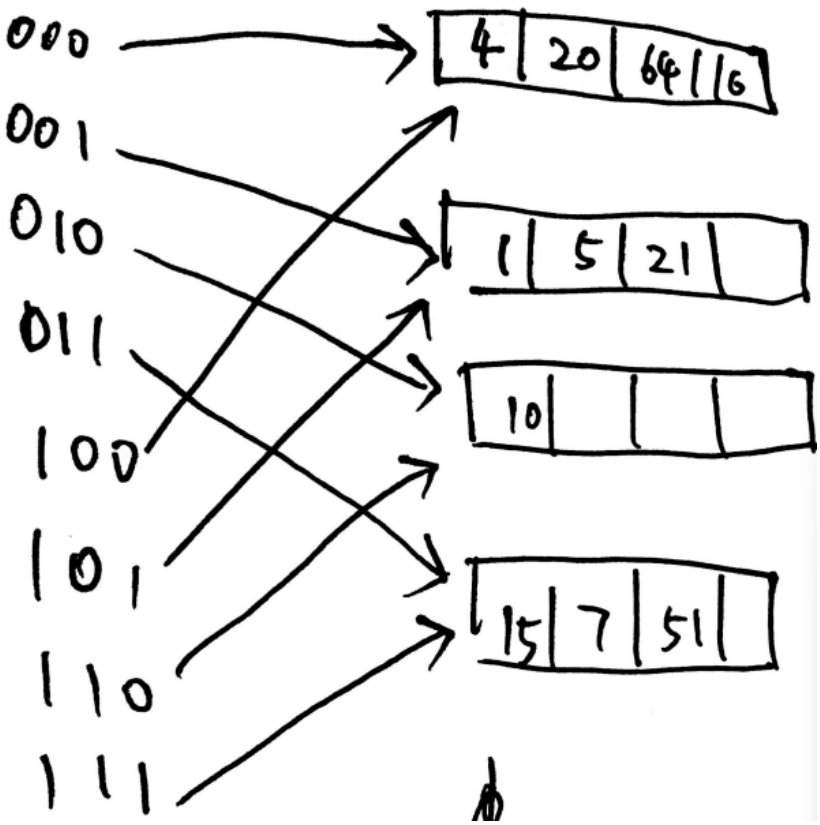
$E_{1,1}$=(W, X), $E_{1,2}$=(W, Y), $E_{1,3}$=(X,Z)

b. **Consider the relational schema R = {P, Q, R, S, T, U, V} and the set of functional dependencies FD: P → Q, Q → R, PS → TRV, QT → UR, S → V. Find its minimal cover.**

P → Q, Q → R, PS → TRV, QT → U, S → V

Q3. [CO 5], [a. – BTL 2, b. – BTL 3, c. – BTL 4], [2+3+3]

a. **Consider a most dense B-trees of order d = 5 that contains all the integer keys 1, 2, . . ., 120. How many nodes does the structure have? -** 12
b. **Consider an extendible hash table, where the last insertion forced its directory to double in size, from n_{old}=8 to n_{new}=16 directory entries. how many buckets have exactly one directory entry pointing to them? -** 2
c. **Starting from the hash table given below, delete keys 36, 12, and plot the resulting table.**



Q4. [CO 3], [BTL 3], [5+3]

a. Following figure gives the schema of bike sharing data collected from five cities in the Bay Area. Write the SQL queries for the following.

i. Count the number of stations in each city. That is print city name and number of stations. Sort by number of stations (decreasing), and break ties by city name (increasing).

```
select city, count(station_id) as cnt
from station
group by city
order by cnt desc, city asc
```

ii. List all stations name of which contains its city's name, for example, station "Mountain View Caltrain Station" in city "Mountain View". That is print station name and city name. Sort by city name (increasing), and break ties by station name (increasing).

```
select station_name, city
from station
where station_name like '%' || city || '%'
order by city asc, station_name asc
```

iii. **Find the count of self-loop trips among all trips. A trip is self-loop when its start station is the same as its end station.**

```
select round(self_loop_cnt.cnt * 1.0 / trip_cnt.cnt, 4)
       as percentage
from (
  select count(*) as cnt
  from trip
  where start_station_id = end_station_id
) as self_loop_cnt,
(
  select count(*) as cnt
  from trip
) as trip_cnt
;
```

b. Consider the relations of a database for the 2016 Olympics as given below. These relations record the athletes, events, and outcomes/results of the 2016 Olympic games.
Athletes: For every athlete, we record a unique athlete id, the country they represent, their name, and their age.
Events: This table lists all the events that are part of the 2016 Olympic games with unique integer event id and a name.
Event Results: Lists the outcomes of all events with the event id of the event, the athlete id of the athlete that won a medal in the event, and the standing of the athlete (i.e., gold, silver or bronze).

athlete_id	country	name	age

- Athletes Table

event_id	name

- Events Table

event_id	athlete_id	result

- Event Results Table

Answer the following-

a. List the names of the athletes that have won at-least one gold medal, eliminating duplicate names, using Relational Algebraic expression.

1. $\pi_{\text{name}}\left(\sigma_{\text{result}='Gold'}\left(\text{Athletes} \bowtie \text{Event\_Results}\right)\right)$
2. $\pi_{\text{name}}\left(\text{Athletes} \bowtie \sigma_{\text{result}='Gold'}\left(\text{Event\_Results}\right)\right)$
3. $\pi_{\text{name}}\left(\sigma_{\text{result}='Gold'}\left(\text{Athletes} \bowtie \pi_{\text{athlete\_id,result}}\left(\text{Event\_Results}\right)\right)\right)$

b. Describe what the following expression does

$$\pi_{\texttt{athlete\_id,event\_id}}(\texttt{Event\_Results}) \div \pi_{\texttt{event\_id}}(\sigma_{\texttt{athlete\_id='A5'}}(\texttt{Event\_Results}))$$

List the athlete ids of all athletes that have won medals in every event that athlete with ID"A5" (i.e., Usain Bolt) has also won a medal

c. Describe what the following expression does

$$\pi_{\texttt{A.athlete\_id}}(\rho_A(\texttt{Athletes})) - \pi_{\texttt{ER1.athlete\_id}}($$
$$\rho_{ER1}(\texttt{Event\_Results}) \bowtie_{ER1.\texttt{athlete\_id}=ER2.\texttt{athlete\_id} \wedge ER1.\texttt{result} \neq ER2.\texttt{result}} \rho_{ER2}(\texttt{Event\_Results}))$$

Finds all the athletes (by athlete id) that did not win two or more types of medal

Q5. [a. c. –CO 3, b. –CO 5, d. – CO 1], [BTL 1], [2+2+2+2]

a. Explain join dependencies with help of an example

Join Dependency (JD) can be illustrated as when the relation R is equal to the join of the sub-relations R1, R2,..., and Rn are present in the database. Join Dependency arises when the attributes in one relation are dependent on attributes in another relation

Join dependency on a database is denoted by: *R1* ⋈ *R2* ⋈ *R3* ⋈ ..... ⋈ *Rn ; where R1 , R2, ... , Rn are the relations and* ⋈ *represents the natural join operator.*

Example of Join Dependency

Suppose you have a table having stats of the company, this can be decomposed into sub-tables to check for the join dependency among them. Below is the depiction of a table Company_Stats having attributes Company, Product, Agent. Then we created sub-tables R1 with attributes Company & Product and R2 with attributes Product & Agent. When we join them we should get the exact same attributes as the original table.

Table: Company_Stats

Company	Product	Agent
C1	TV	Aman
C1	AC	Aman
C2	Refrigerator	Mohan
C2	TV	Mohit

Table: R1

Company	Product
C1	TV
C1	AC
C2	Refrigerator
C2	TV

Table: R2

Product	Agent
TV	Aman
AC	Aman
Refrigerator	Mohan
TV	Mohit

On performing join operation between R1 & R2: R1 ⋈ R2

Company	Product	Agent
C1	TV	Aman
C1	TV	Mohan
C1	AC	Aman
C2	Refrigerator	Mohan
C2	TV	Aman
C2	TV	Mohit

Here, we can see that we got two additional tuples after performing join i.e. (C1, TV, Mohan) & (C2, TV, Aman) these tuples are known as Spurious Tuple, which is not the property of Join Dependency. Therefore, we will create another relation R3 and perform its natural join with (R1 ⋈ R2). So, here it is:

Table: R3

Company	Agent
C1	Aman
C2	Mohan
C2	Mohit

Now on doing natural join of (R1 ⋈ R2) ⋈ R3, we get

Company	Product	Agent
C1	TV	Aman
C1	AC	Aman
C2	Refrigerator	Mohan
C2	TV	Mohit

Now, we got our original relation, that we had earlier decomposed, in this way you can decompose the original relation and check for the join dependency among them. Importance of Join Dependencies
Join dependency can be very important for several reasons as it helps in maintaining data integrity, possess normalization, helps in query optimization within a database.

b. Differentiate between primary index and Clustering index.

Feature	Primary Index	Clustering Index
Definition	A unique identifier for each record in a table	A special type of index that reorders the way records in a table are physically stored
Purpose	Ensures data integrity and uniqueness	Improves performance of queries that involve sorting or range searches
Uniqueness	Must be unique	Can be unique or non-unique
Number per Table	Only one per table	Only one per table
Data Storage	Doesn't directly affect physical storage	Physically reorders data on disk
Performance Impact	Improves query performance, especially for equality comparisons	Significantly improves performance for range queries and sorting

Key Differences:

- Data Organization: A primary index doesn't directly affect the physical storage of data, while a clustering index reorders the physical storage of data based on the index key.
- Uniqueness: A primary index must be unique, while a clustering index can be unique or non-unique.

- Performance Impact: Both improve query performance, but a clustering index is particularly effective for range queries and sorting.

c. Define Natural Join with the help of an example

A natural join is a type of join operation in relational databases that combines rows from two or more tables based on a common column(s) with the same name and data type. The resulting table includes all columns from both tables, but with only one occurrence of each common column.

Example: Consider two tables:

Table: Students

Student_ID	Student_Name	Department
101	Alice	Computer Science
102	Bob	Electrical Engineering
103	Charlie	Computer Science

Table: Courses

Course_ID	Course_Name	Department
C101	Database Systems	Computer Science
C102	Digital Logic	Electrical Engineering
C103	Operating Systems	Computer Science

To find the courses each student is enrolled in, we can perform a natural join on the Department columnusing SQL as

```
SELECT *
FROM Students
NATURAL JOIN Courses;
```

This query will produce the following result:

Student_ID	Student_Name	Department	Course_ID	Course_Name
101	Alice	Computer Science	C101	Database Systems
101	Alice	Computer Science	C103	Operating Systems
102	Bob	Electrical Engineering	C102	Digital Logic
103	Charlie	Computer Science	C101	Database Systems
103	Charlie	Computer Science	C103	Operating Systems

As you can see, the natural join automatically matches rows based on the common Department column and combines the information from both tables.

d. **State True/ False and justify your answer**
    a. **In the shrinking phase of strict 2PL, locks cannot be released until the end of the transaction. -** Yes
    b. **Only schedules under 2PL (and not strict 2PL) may lead to deadlocks.** No

Q6 [CO2], [a - BTL 5, b - BTL 1], [5+3]

a. **Consider a database to store information for a social networking website. Design an ER Diagram for the database having following properties:**

Every user has a unique user ID (integer) along with a full name, age and phone number. Every group has a unique group ID (integer) and a name. Every group must have at least one user that serves as moderator of the group. A user may be a member of zero or more groups; groups may contain zero or more members (and one or more moderators). Users are allowed to create zero or more albums. An album has a unique album ID (integer), a creation date, and a name. An album is owned by exactly one user: the user that created it. • An album can contain zero or more media files. For every media file, we record its unique URL, the date the file was added to the album, and a caption (if one exists). A media file may belong to at most one album.

b. Explain the Characteristics of a Database Approach for a given application

A number of characteristics distinguish the database approach from the much older approach of writing customized programs to access data stored in files. In traditional file processing, each user defines and implements the files needed for a specific software application as part of programming the application. For example, one user, the *grade reporting office*, may keep files on students and their grades. Programs to print a student's transcript and to enter new grades are implemented as part of the application. A second user, the *accounting office*, may keep track of students' fees and their payments. Although both users are interested in data about students, each user maintains separate files—and programs to manipulate these files—because each requires some data not available from the other user's files. This redundancy in defining and storing data results in wasted storage space and in redundant efforts to maintain common up-to-date data.

In the database approach, a single repository maintains data that is defined once and then accessed by various users repeatedly through queries, transactions, and application programs. The main characteristics of the database approach versus the file-processing approach are the following:

- Self-describing nature of a database system
- Insulation between programs and data, and data abstraction
- Support of multiple views of the data
- Sharing of data and multiuser transaction processing

X---------END----------X